

Examen de fin de semestre SRCS Master 1 SAR

Jonathan Lejeune

Mai 2021

2 heures Tout document papier interdit
Tout appareil électronique interdit

- Le barème est sur 22 points et est donné à titre indicatif. La note finale sera bornée à 20 points.
- Dans le code java demandé, vous ne gérerez pas les imports ni les exceptions sauf si c'est explicitement demandé
- Soyez synthétique et précis dans vos réponses
- Il vous est conseillé de lire attentivement les exercices entièrement avant de composer.
- Lorsque vous ne savez pas répondre à une question, il est préférable de s'abstenir que de répondre une absurdité.

Exercice 1 – Questions de cours (8 points)

- Question 1 1 point
Dans le paradigme des objets distants, qu'est-ce qu'un proxy? *Proxymon intermédiaire*
- Question 2 1 point
En RMI, comment est géré le transfert d'un objet qui est à la fois Remote et à la fois Serializable?
- Question 3 1 point
Dans quel cas la méthode distante bind du RMI Registry lance une ClassNotFoundException?
- Question 4 1/2 point
Donner une différence qu'il y a entre un objet et un composant.
- Question 5 1 point
Donner le nom des parties qui composent un URI.
- Question 6 1 point
Dans le modèle du web, qu'est-ce qu'un proxy? À quoi sert-il? *serveur intermédiaire*
- Question 7 1 1/2 points
Donner les 6 contraintes pour qu'un protocole soit considéré Restful.
- Question 8 1 point
Expliquer les étapes de la vérification d'une signature numérique utilisant un algorithme asymétrique
*la signature par elle-même authentifiée
et bien authentifiée*

Exercice 2 – Allocation de conteneurs (14 points)

Un datacenter est composé de machines mises à disposition d'un ensemble d'utilisateurs pour qu'ils puissent y déployer des calculs. Afin d'en partager équitablement l'utilisation, ces machines sont conteneurisées. La conteneurisation est un outil offert par les systèmes d'exploitation actuels. Ceci consiste à allouer exclusivement une quantité de ressources de calcul d'une machine à un utilisateur donné. Dans le cadre de cet exercice, on entend par ressources de calcul les cœurs du processeur et la mémoire RAM.

Afin d'orchestrer l'ensemble de ces ressources, on considère :

- que chaque machine possède un service RMI (MachineManager) qui contrôle les ressources locales en allouant ou en détruisant des conteneurs. Ce service est donc chargé de maintenir le nombre de ressources disponibles (abcore et nbram) d'une machine. Le nombre de ressources totales sera renseigné à la construction du service et d'offrir les services suivants :
 - ◊ connaître le nombre de ressources disponibles
 - ◊ allouer un nouveau conteneur sur la machine
 - ◊ désallouer un conteneur existant sur la machine
 - ◊ permettre à l'utilisateur propriétaire d'un conteneur existant sur la machine d'y exécuter une commande (on supposera que le code de la commande est déjà disponible sur la machine hôte).
- l'existence d'une machine maître sur laquelle est déployé exclusivement un service RMI (DCManager). Le rôle de ce service est de maintenir une liste de MachineManager disponibles (qui sera renseignée à la construction du service) et qui offrira aux utilisateurs la possibilité d'allouer un nouveau conteneur sur le datacenter.

Ainsi, lorsqu'un utilisateur du datacenter souhaite allouer un conteneur, il s'adresse au DCManger en renseignant la quantité de ressources qu'il souhaite allouer c'est à dire une quantité de cœurs et une quantité de RAM en Go. À la réception de la requête, le DCManger contrôle s'il existe une machine qui peut héberger ce conteneur :

- Dans le cas où c'est possible, le DCManger notifie le MachineManager correspondant d'allouer les ressources demandées pour l'utilisateur en question. Une fois les ressources allouées sur la machine, le DCManger répond au client en retournant deux informations :
 - ◊ le MachineManager sur lequel le conteneur a été alloué
 - ◊ un identifiant de conteneur (de type entier) qui doit être pouvoir identifier de manière unique un conteneur dans le temps et sur l'ensemble du datacenter.
 - L'utilisateur peut alors contacter le MachineManager pour exécuter la commande de son choix dans le conteneur en question (on supposera que le code de la commande est déjà disponible sur la machine hôte).
 - Dans le cas où le conteneur ne peut être alloué par manque de ressources disponibles, le DCManger notifiera l'utilisateur via une exception ResourceNotAvailableException.
- On considérera :
- un système fiable (pas de panne de machine et pas de pertes de messages) ✓
 - qu'une requête ne concerne qu'un seul conteneur ✓
 - qu'il est possible que des conteneurs alloués appartiennent au même utilisateur. ✓
 - que l'on dispose des méthodes statiques OS.createContainer(...) OS.ExecuteInContainer(...) et OS.destroyContainer(...) qui permettent respectivement de 1) d'allouer physiquement un conteneur 2) d'exécuter physiquement du code dans un conteneur existant et 2) de désallouer physiquement un conteneur existant (et de tuer le programme qui s'y exécute le cas échéant). Les paramètres et le retour de ces méthodes sont à définir par vos soins en fonction de vos choix de conception.

Lorsqu'un utilisateur souhaite supprimer un conteneur, il enverra une requête au DCManger qui contactera le MachineManager concerné pour que ce dernier tue le conteneur et ainsi libérer les

ressources associées.

Question 1 6 points

Donner le code nécessaire pour mettre en place ce service. Il est attendu :

- des interfaces RMI (vous commenterez chaque méthode par sa spécification : rôle, sémantique des paramètres, sémantique du retour)
 - des classes d'implantations des services RMI
 - le cas échéant des classes java définissant des messages
 - ✓ la classe `ResourceNotAvailableException`
- Les accès concurrents sur des variables partagées doivent être correctement gérés.

Question 2 3 points

Afin d'éviter qu'un utilisateur monopolise trop longtemps les ressources du datacenter, on souhaite à présent enrichir l'API de réservation en renseignant un temps maximum d'utilisation, c'est-à-dire que passé ce délai, le conteneur sera automatiquement désalloué. Ainsi, au moment de réserver le `DCManager` utilisera un mécanisme d'armement de timer. À l'échéance de ce timer, le `DCManager` enverra une requête de désallocation au `MachineManager` correspondant. Indiquer les modifications à appliquer sur votre code. 2

Question 3 2 points

On souhaite pouvoir notifier l'utilisateur quand le temps d'utilisation d'un de ces conteneurs est arrivé à échéance. Indiquer les modifications à appliquer à la fois sur les interfaces et sur les classes d'implantations pour respecter cette spécification.

Question 4 3 points

Nous considérons à présent que le `DCManager` maintient une liste de clés publiques d'utilisateurs autorisés et reconnus. Nous souhaitons assurer les 3 spécifications suivantes :

1. le système ne peut être accessible qu'à ces utilisateurs.
2. les opérations d'allocation et de destruction de conteneur sur une machine sont exclusivement initiées par le `DCManager`
3. un utilisateur ne peut exécuter une commande que dans les conteneurs qui lui appartiennent.

Dessiner sur un diagramme les échanges de messages d'un protocole (basé sur l'utilisation de clé cryptographique) qui permet d'assurer ces 3 spécifications. Pour chaque spécification, vous justifierez en quoi votre solution permet de la respecter.